Практичні приклади до уроку 8.2 - Error Tracking та аналіз помилок: Sentry і продакшн-практики

> **Загальна ідея практики:**
> Показати, як **підключити Sentry** до Laravel-проєкту, які помилки туди відправляти, як додавати **фінансовий контекст** (user_id, payment_id, account_id, gateway_code), як аналізувати issues в інтерфейсі Sentry і як Sentry вписується в сценарії інцидентів.
> Приклади відрізняються від урочних фрагментів, але сумісні з архітектурою з попередніх модулів.

1. Блок 3: Інтеграція Sentry → Слайд 8a

Загальна ідея прикладу:
Мінімально інтегрувати Sentry у Laravel-застосунок: **встановлення, конфігурація DSN, перевірка**, що базові винятки долітають.

1.1. Встановлення пакета

```
composer require sentry/sentry-laravel
```

1.2. Налаштування DSN та середовища

У `.env`:

```
SENTRY_LARAVEL_DSN=https://<key>@sentry.io/<project-id>
SENTRY_TRACES_SAMPLE_RATE=0.0
SENTRY_PROFILES_SAMPLE_RATE=0.0
```

У `config/app.php` (якщо потрібно - для старих версій Laravel) переконатися, що провайдер доданий, або покластися на авто-дискавер:

```
Sentry\Laravel\ServiceProvider::class,
```

У `config/sentry.php` (після `php artisan vendor:publish --provider="Sentry\Laravel\ServiceProvider"`):

```
return [
    'dsn' => env('SENTRY_LARAVEL_DSN'),
    'environment' => env('APP_ENV', 'production'),
    'error_types' => E_ALL & ~E_DEPRECATED & ~E_USER_DEPRECATED,
```

```
    'breadcrumbs' => [
        'logs' => true,
        'sql_queries' => true,
        'sql_bindings' => true,
        'queue_info' => true,
    ],

    'send_default_pii' => false,
];
```

1.3. Тестовий ендпоінт для перевірки

```php
<?php

declare(strict_types=1);

use Illuminate\Support\Facades\Route;

Route::get('/api/v1/test-sentry', static function (): void {
    throw new RuntimeException('Test Sentry integration');
});
```

Після запиту до `/api/v1/test-sentry` відповідний exception має з'явитися в Sentry у вибраному проекті, з environment, що відповідає `APP_ENV`.

1.4. Що вже реалізовано в цьому проєкті

- Додано endpoint `GET /api/v1/test-sentry` у `routes/api.php` (throw `RuntimeException('Test Sentry integration')`).
- Для сумісності з попереднім демо залишено також `GET /api/v1/sentry-test`.
- Middleware `AssignCorrelationId` не лише додає `X-Correlation-ID` у request/response і в логи, а й пробрасывает `correlation_id` та `endpoint` у Sentry scope.

Швидка перевірка:

```
curl -i http://localhost/api/v1/test-sentry -H "X-Correlation-ID: demo-sentry-001"
```

> **Що проговорити на демонстрації:**
>
> - що SDK автоматично ловить необработанные exceptions;
> - як `APP_ENV` впливає на поле environment у Sentry;
> - що далі треба налаштовувати *які* помилки й з яким контекстом слати, а не все підряд.

2. Блок 4: reportPaymentFailure → Слайд 11a

Загальна ідея прикладу:
Реалізувати допоміжний метод `reportPaymentFailure(...)`, який одночасно **логирує помилку** і **відправляє її в Sentry** з мінімально необхідним фінансовим контекстом.

2.1. Сервіс reportPaymentFailure в Laravel

```php
<?php

declare(strict_types=1);

namespace App\Services\Monitoring;

use Illuminate\Support\Facades\Log;
use RuntimeException;

final class PaymentErrorReporter
{
    public function reportPaymentFailure(
        int $paymentId,
        int $userId,
        string $gatewayCode,
        string $correlationId
    ): void {
        $message = 'Payment processing failed at gateway level';

        // 1. Структурований лог
        Log::error($message, [
            'payment_id' => $paymentId,
            'user_id' => $userId,
            'gateway_code' => $gatewayCode,
            'correlation_id' => $correlationId,
        ]);

        // 2. Відправка в Sentry з контекстом
        \Sentry\withScope(function (\Sentry\State\Scope $scope) use
($paymentId, $userId, $gatewayCode, $correlationId, $message): void {
            $scope->setTag('module', 'payments');
            $scope->setTag('action', 'gateway_failure');
            $scope->setTag('gateway_code', $gatewayCode);
            $scope->setUser(['id' => (string) $userId]);

            $scope->setExtra('payment_id', $paymentId);
            $scope->setExtra('correlation_id', $correlationId);

            \Sentry\captureException(new RuntimeException($message));
        });
    }
}
```

> **Що проговорити:**

> - тут ми не пересылаем оригинальный exception шлюза (если он уже обработан), а создаём свой с контекстом;
> - `gateway_code` и `module=payments` в `tags` позволяют быстро фильтровать ошибки в Sentry;
> - `payment_id`, `user_id`, `correlation_id` в `extra`/user позволяют перейти к логам и данным в БД.

2.2. Де саме викликається `PaymentErrorReporter` у цьому проєкті

1. **Основний виклик у бойовому потоці:**
   `app/Http/Controllers/Api/V1/PaymentController.php`, метод `store()`:
   - при `catch (Throwable $exception)` викликається `reportPaymentFailure(...)`;
   - після цього exception пробрасывается далі (щоб не змінювати поточний контракт помилок API).
2. **Безпечний demo-виклик для уроку:**
   Додано endpoint `POST /api/v1/payments/demo-fail` (`PaymentController@demoFail`), який:
   - явно викликає `PaymentErrorReporter`;
   - повертає `202` і `correlation_id`;
   - не ламає стандартний сценарій створення платежу.
3. **Реалізований клас репортера:**
   `app/Services/Monitoring/PaymentErrorReporter.php`.

2.3. Команда для демо-тригера `PaymentErrorReporter`

```
curl -i -X POST http://localhost/api/v1/payments/demo-fail \
  -H "X-Correlation-ID: demo-per-002"
```

Очікуваний результат:

- HTTP `202 Accepted`;
- у логах є `Payment processing failed at gateway level`;
- у Sentry є подія з tags:
  - `module=payments`
  - `action=gateway_failure`
  - `gateway_code=DEMO_FAIL`

3. Блок 5: Аналіз issue → Слайд 14a

Загальна ідея прикладу:
Описати, як виглядає **типове issue в Sentry** для `PaymentGatewayException` і як, ґрунтуючись на ньому, приймати рішення.

3.1. Умовний Sentry-issue “PaymentGatewayException”

- **Title:** `PaymentGatewayException: Gateway responded with HTTP 500`
- **Environment:** `production`
- **Events:** 120 за останню годину
- **Users:** 45 affected users
- **First seen:** сьогодні о 10:05
- **Last seen:** сьогодні о 11:00
- **Release:** `payments-service@1.3.0`
- **Tags:**
  - `module: payments`
  - `action: capture`
  - `gateway: some_gateway`
  - `gateway_code: INTERNAL_ERROR`
- **Typical extra:**
  - `payment_id: 12345`
  - `account_id: 10`
  - `amount: "250.00"`

3.2. Висновки з цього issue

- Проблема **масова** (120 подій, 45 користувачів) → високий пріоритет.
- Почалась **після релізу 1.3.0** → можлива регресія нового коду або зміни на стороні шлюзу, що збіглися з релізом.
- Всі помилки - при `action=capture` → проблема на етапі підтвердження платежу.
- `gateway_code=INTERNAL_ERROR` → збій на стороні провайдера або некоректний payload з нашого боку.

3.3. Наступні кроки команди

1. Перевірити, чи не змінювали payload/формат параметрів у релізі 1.3.0 для `capture`.
2. Переглянути логи по `correlation_id` одного з проблемних платежів (зв'язок Sentry ↔ лог-агрегація).
3. Зв'язатися з платіжним провайдером (якщо проблема не в payload), надати `gateway_code`, час інциденту та кілька `payment_id`.
4. За потреби - тимчасово переключитися на резервний шлюз або вимкнути capture для певних кейсів.

> **Що підкреслити на демонстрації:**
>
> - як завдяки tags/extra issue відразу дає зрозуміти, *де* і *як часто* це відбувається;
> - як прив'язка до release дозволяє зрозуміти, це регресія чи "старий" баг;
> - як issue стає основою для прийняття рішень (hotfix/rollback/зв'язок із провайдером).

4. Блок 7: Сценарій інциденту з релізом → Слайд 19а

Загальна ідея прикладу:
Показати, як error tracking використовується в **повному сценарії інциденту** після релізу платіжного модуля.

4.1. Опис сценарію

> Після деплою релізу `payments-service@2.0.0` команда бачить у Sentry сплеск нового issue `InsufficientBalanceException` на endpoint `POST /api/v1/payments`. Кількість подій різко зросла: раніше - 5/день, тепер - 200/годину.

4.2. Аналіз issue

- Перегляд графіка: різкий стрибок після релізу 2.0.0.
- Перевірка **affected users**: багато різних користувачів → не локальна проблема з одним акаунтом.
- Контекст: в `extra` видно, що `available_balance` близько до нуля, але новий розрахунок ліміту став суворіший (наприклад, не допускає залишок менше 50).

4.3. Рішення

1. З'ясувати, чи зміни в логіці перевірки балансу були **навмисними** (новий бізнес-rule) чи це **баг**.
2. Якщо це баг - швидкий **hotfix** або **rollback** релізу.
3. Якщо це нове правило - можливо, занадто агресивне → погодити з бізнесом коригування (наприклад, мінімальний залишок зменшити).

4.4. Post-mortem

- Документувати, що зміна перевірки балансу без чіткої оцінки впливу призвела до масових відмов.
- Додати правило: будь-яка зміна бізнес-правил у критичних модулях (баланс/комісії) має супроводжуватися:
  - оновленням тестів (unit + API);
  - очікуваним впливом (кількість користувачів, для яких зміниться поведінка);
  - відстеженням issue в Sentry після релізу.

> **Що проговорити:**
>
> - як Sentry допоміг побачити не просто "помилку", а саме **сплеск** певного типу;
> - як це зв'язується з релізом і тестами, про які ви говорили в Модулі 6–7.

5. Примітки щодо середовища / docker-compose

- У docker-середовищі Sentry вимагає лише наявності доступу з контейнера `app` до інтернету (або до self-hosted Sentry, якщо ви його розгортаєте).
- DSN і APP_ENV передаються через `.env` у контейнері.
- Налаштуйте мінімальне sampling (або фільтрацію типів винятків) для середовищ з дуже великим потоком помилок, щоб не перевантажувати Sentry шумом.

> **Головна ідея практичної частини уроку 8.2:**
> Error tracking (на прикладі Sentry) - це **системний шар роботи з помилками**, а не просто "ще одне місце, куди щось летить". Правильно обравши, *які* помилки відправляти і *який контекст* до них додавати, ви перетворюєте хаотичні збої на керований список issues з пріоритетами, історією та прив'язкою до релізів. Це особливо важливо в фінансових застосунках, де кожна помилка може впливати на гроші користувачів.